

LAB EXPERIMENTS:TOPIC 1

Experiment 1: First Palindromic String in an Array

Aim

To write a program that returns the first palindromic string from a given array of strings, or an empty string if none exists.

Algorithm

- Start.
- Read the array of strings words.
- For each string in words (in order), check whether it reads the same forward and backward.
- If a palindromic string is found, return it immediately and stop.
- If no string in the array is palindromic, return an empty string "".
- Stop.

Pseudocode

```
FUNCTION firstPalindrome(words):  
  FOR each word IN words:  
    IF word == reverse(word):  
      RETURN word  
  RETURN ""
```

Sample Input & Output

Sample Input	words = ["abc","car","ada","racecar","cool"]
---------------------	--

Sample Output	"ada"
----------------------	-------

Time & Space Complexity

Time Complexity	$O(n * m)$, where n is the number of strings and m is the average length of a string (each string may need to be reversed/compared).
------------------------	---

Space Complexity	$O(1)$ extra space ($O(m)$ if a reversed copy of the string is created).
-------------------------	---

Result

The program was executed and the first palindromic string was correctly identified for the given test cases.

Experiment 2: Count of Common Elements Between Two Arrays

Aim

To determine, for two integer arrays `nums1` and `nums2`, the number of indices in `nums1` whose value exists in `nums2` and vice versa.

Algorithm

- Start.
- Read arrays `nums1` and `nums2`.
- Convert `nums2` into a hash set `set2` for $O(1)$ lookups; similarly build `set1` from `nums1`.
- Initialize `answer1 = 0`, `answer2 = 0`.
- For each element in `nums1`, if it is present in `set2`, increment `answer1`.
- For each element in `nums2`, if it is present in `set1`, increment `answer2`.
- Return `[answer1, answer2]`.
- Stop.

Pseudocode

```
FUNCTION countCommon(nums1, nums2):  
    set1 = SET(nums1)  
    set2 = SET(nums2)  
    answer1 = COUNT(x IN nums1 WHERE x IN set2)  
    answer2 = COUNT(x IN nums2 WHERE x IN set1)  
    RETURN [answer1, answer2]
```

Sample Input & Output

Sample Input	<code>nums1 = [4,3,2,3,1]</code> , <code>nums2 = [2,2,5,2,3,6]</code>
---------------------	---

Sample Output	<code>[3,4]</code>
----------------------	--------------------

Time & Space Complexity

Time Complexity	$O(n + m)$, where $n = \text{nums1} $ and $m = \text{nums2} $ (building sets and scanning each array once).
------------------------	--

Space Complexity	$O(n + m)$ for the two hash sets.
-------------------------	-----------------------------------

Result

The program correctly computed `answer1` and `answer2` for the given test cases, matching the expected output.

Experiment 3: Sum of Squares of Distinct Counts of All Subarrays

Aim

To compute the sum of the squares of the number of distinct elements over every contiguous subarray of a given array.

Algorithm

- Start.
- Read the array nums of length n.
- Initialize totalSum = 0.
- For every starting index i from 0 to n-1, initialize an empty set/frequency map and reset distinctCount = 0.
- For every ending index j from i to n-1, add nums[j] to the set; if it is newly added, increment distinctCount.
- Add (distinctCount)^2 to totalSum.
- After both loops finish, return totalSum.
- Stop.

Pseudocode

```
FUNCTION sumOfSquares(nums):  
  n = LENGTH(nums)  
  totalSum = 0  
  FOR i = 0 TO n-1:  
    seen = EMPTY SET  
    FOR j = i TO n-1:  
      seen.ADD(nums[j])  
      totalSum += (SIZE(seen))^2  
  RETURN totalSum
```

Sample Input & Output

Sample Input	nums = [1,2,1]
--------------	----------------

Sample Output	15
---------------	----

Time & Space Complexity

Time Complexity	$O(n^2)$ — two nested loops over all subarrays; set insertion is $O(1)$ amortized.
-----------------	--

Space Complexity	$O(n)$ for the set used to track distinct elements of the current subarray.
------------------	---

Result

The program correctly enumerated all subarrays and computed the sum of squares of distinct counts, matching the expected outputs 15 and 3.

Experiment 4: Count of Equal-Value Pairs with Index Product Divisible by k

Aim

To count pairs (i, j) with $i < j$ such that $\text{nums}[i] == \text{nums}[j]$ and $(i * j)$ is divisible by k.

Algorithm

- Start.
- Read array nums of length n and integer k.
- Initialize count = 0.
- For every pair of indices i from 0 to n-2 and j from i+1 to n-1:
 - If $\text{nums}[i] == \text{nums}[j]$ AND $(i * j) \% k == 0$, increment count.
- Return count.
- Stop.

Pseudocode

```
FUNCTION countPairs(nums, k):  
  n = LENGTH(nums)  
  count = 0  
  FOR i = 0 TO n-2:  
    FOR j = i+1 TO n-1:  
      IF  $\text{nums}[i] == \text{nums}[j]$  AND  $(i*j) \text{ MOD } k == 0$ :  
        count += 1  
  RETURN count
```

Sample Input & Output

Sample Input	nums = [3,1,2,2,2,1,3], k = 2
---------------------	-------------------------------

Sample Output	4
----------------------	---

Time & Space Complexity

Time Complexity	$O(n^2)$ for the brute-force double loop over all index pairs.
------------------------	--

Space Complexity	$O(1)$ extra space.
-------------------------	---------------------

Result

All 4 valid pairs were correctly identified for the given test case, matching the expected output.

Experiment 5: Find the Maximum Element in a List (Least Time Complexity)

Aim

To design a program that finds the maximum element of a list using the least possible time complexity.

Algorithm

- Start.
- Read the list of numbers.
- If the list is empty, report an error / return None.
- Initialize maxVal = first element of the list.
- Traverse the list once; for every element, if element > maxVal, update maxVal = element.
- After the traversal, return maxVal.
- Stop.

Pseudocode

```
FUNCTION findMax(list):  
  IF list IS EMPTY: RETURN None  
  maxVal = list[0]  
  FOR i = 1 TO LENGTH(list)-1:  
    IF list[i] > maxVal:  
      maxVal = list[i]  
  RETURN maxVal
```

Sample Input & Output

Sample Input	{1, 2, 3, 4, 5} {7, 7, 7, 7, 7} {-10, 2, 3, -4, 5}
---------------------	--

Sample Output	5 7 5
----------------------	-----------

Time & Space Complexity

Time Complexity	$O(n)$ — a single linear scan is the least possible time complexity since every element must be examined at least once.
------------------------	---

Space Complexity	$O(1)$ extra space.
-------------------------	---------------------

Result

The program correctly returned 5, 7, and 5 respectively for the three test cases, achieving optimal $O(n)$ time.

Experiment 6: Sort a List and Find the Maximum Element

Aim

To sort a list of numbers using an efficient sorting algorithm and then find the maximum element from the sorted list.

Algorithm

- Start.
- Read the input list.
- If the list is empty, return None with an appropriate message.
- Sort the list using an efficient comparison sort (e.g., Merge Sort / built-in Timsort), which runs in $O(n \log n)$.
- Return the last element of the sorted list as the maximum.
- Stop.

Pseudocode

```
FUNCTION sortAndFindMax(list):  
  IF list IS EMPTY: RETURN "List is empty"  
  sortedList = SORT(list) //  $O(n \log n)$   
  RETURN sortedList[LAST INDEX]
```

Sample Input & Output

Sample Input	[] [5] [3,3,3,3,3]
---------------------	------------------------

Sample Output	"List is empty" 5 3
----------------------	-------------------------

Time & Space Complexity

Time Complexity	$O(n \log n)$, dominated by the sorting step.
------------------------	--

Space Complexity	$O(n)$ auxiliary space for merge sort (or $O(\log n)$ if an in-place $O(n \log n)$ sort is used).
-------------------------	---

Result

The program correctly handled the empty list, single-element list, and list of identical elements, returning the expected maximum in each case.

Experiment 7: Extract Unique Elements from a List

Aim

To write a program that creates a new list containing only the unique elements from an input list, and to analyze its space complexity.

Algorithm

- Start.
- Read the input list of n numbers.
- Initialize an empty hash set `seen` and an empty result list.
- For each element in the input list, if it is not already in `seen`, add it to `seen` and append it to the result list.
- Return the result list.
- Stop.

Pseudocode

```
FUNCTION uniqueElements(list):  
    seen = EMPTY SET  
    result = EMPTY LIST  
    FOR x IN list:  
        IF x NOT IN seen:  
            seen.ADD(x)  
            result.APPEND(x)  
    RETURN result
```

Sample Input & Output

Sample Input	[3, 7, 3, 5, 2, 5, 9, 2]
---------------------	--------------------------

Sample Output	[3, 7, 5, 2, 9]
----------------------	-----------------

Time & Space Complexity

Time Complexity	$O(n)$ — a single pass through the list with $O(1)$ average-case set operations.
------------------------	--

Space Complexity	$O(n)$ — in the worst case (all elements unique) the hash set and result list each hold n elements.
-------------------------	---

Result

The program correctly extracted unique elements while preserving first-occurrence order, for all three test cases.

Experiment 8: Bubble Sort

Aim

To sort an array of integers using the Bubble Sort technique and analyze its time complexity.

Algorithm

- Start.
- Read the array of n integers.
- Repeat for i = 0 to n-2:
 - For j = 0 to n-2-i, compare adjacent elements arr[j] and arr[j+1].
 - If arr[j] > arr[j+1], swap them.
- Continue until no swaps are needed in a full pass (array is sorted).
- Return the sorted array.
- Stop.

Pseudocode

```
FUNCTION bubbleSort(arr):  
  n = LENGTH(arr)  
  FOR i = 0 TO n-2:  
    FOR j = 0 TO n-2-i:  
      IF arr[j] > arr[j+1]:  
        SWAP(arr[j], arr[j+1])  
  RETURN arr
```

Sample Input & Output

Sample Input	[64, 25, 12, 22, 11]
---------------------	----------------------

Sample Output	[11, 12, 22, 25, 64]
----------------------	----------------------

Time & Space Complexity

Time Complexity	Worst/Average case $O(n^2)$; Best case $O(n)$ if optimized with an early-exit flag on an already-sorted array.
------------------------	---

Space Complexity	$O(1)$ — sorting is done in place.
-------------------------	------------------------------------

Result

The array was successfully sorted in ascending order; the $O(n^2)$ behaviour was observed for unsorted inputs.

Experiment 9: Binary Search

Aim

To check whether a given key exists in a sorted array using Binary Search and to analyze its time complexity.

Algorithm

- Start.
- Read sorted array arr and key x.
- Initialize low = 0, high = n-1.
- While low <= high:
 - Compute mid = (low + high) / 2.
 - If arr[mid] == x, return mid (found).
 - Else if arr[mid] < x, set low = mid + 1.
 - Else set high = mid - 1.
- If the loop ends without finding x, return -1 (not found).
- Stop.

Pseudocode

```
FUNCTION binarySearch(arr, x):  
  low = 0; high = LENGTH(arr) - 1  
  WHILE low <= high:  
    mid = (low + high) / 2  
    IF arr[mid] == x: RETURN mid  
    ELSE IF arr[mid] < x: low = mid + 1  
    ELSE: high = mid - 1  
  RETURN -1
```

Sample Input & Output

Sample Input	arr = {3,4,6,-9,10,8,9,30} (sorted view), KEY = 10
---------------------	--

Sample Output	Element 10 is found at position 5
----------------------	-----------------------------------

Time & Space Complexity

Time Complexity	$O(\log n)$ — the search interval halves at every step.
------------------------	---

Space Complexity	$O(1)$ for the iterative version ($O(\log n)$ call-stack space for a recursive version).
-------------------------	---

Result

The key 10 was located correctly; the key 100 (absent) correctly reported 'not found', confirming $O(\log n)$ performance.

Experiment 10: Sort an Array Without Built-in Functions in $O(n \log n)$ Time

Aim

To sort an array of integers in ascending order in $O(n \log n)$ time and with the smallest possible space complexity, without using built-in sort functions.

Algorithm

- Start.
- Read the array nums.
- If the array has 0 or 1 elements, it is already sorted; return it.
- Otherwise, divide the array into two halves (Merge Sort).
- Recursively sort the left half and the right half.
- Merge the two sorted halves into a single sorted array using a linear merge procedure.
- Return the fully merged, sorted array.
- Stop.

Pseudocode

```
FUNCTION mergeSort(nums):  
  IF LENGTH(nums) <= 1: RETURN nums  
  mid = LENGTH(nums) / 2  
  left = mergeSort(nums[0:mid])  
  right = mergeSort(nums[mid:])  
  RETURN merge(left, right)
```

```
FUNCTION merge(left, right):  
  result = EMPTY LIST  
  WHILE left AND right NOT EMPTY:  
    IF left[0] <= right[0]: result.APPEND(left.POP_FRONT())  
    ELSE: result.APPEND(right.POP_FRONT())  
  result.APPEND(remaining of left/right)  
  RETURN result
```

Sample Input & Output

Sample Input	nums = [5, 2, 3, 1]
---------------------	---------------------

Sample Output	[1, 2, 3, 5]
----------------------	--------------

Time & Space Complexity

Time Complexity	$O(n \log n)$ in all cases — the array is split $\log n$ times and merged in $O(n)$ at every level.
------------------------	---

Space Complexity	$O(n)$ auxiliary space for the merge buffers (this is the smallest achievable for a stable $O(n \log n)$ sort without built-ins; an in-place variant such as heap sort achieves $O(1)$ extra space but is not stable).
-------------------------	--

Result: The custom merge sort correctly produced the ascending sorted array in $O(n \log n)$ time.

Experiment 11: Number of Ways a Ball Can Move Out of a Grid in Exactly N Steps

Aim

To find the number of ways a ball starting at cell (i, j) in an m x n grid can be moved out of the grid boundary in exactly N steps.

Algorithm

- Start.
- Read m, n, N, and starting position (i, j).
- Define $dp[step][r][c]$ = number of ways the ball can be out of bounds after 'step' moves starting from (r, c).
- Base case: $dp[0][r][c] = 0$ for all valid (r, c) (0 steps cannot move out).
- For step = 1 to N: for every cell (r, c), for every of the 4 directions, if the neighbour is out of the grid, add 1 way; otherwise add $dp[step-1][neighbour_r][neighbour_c]$.
- The answer is $dp[N][i][j]$, the number of ways starting at (i, j) after exactly N steps.
- Return the answer modulo 10^9+7 if required.
- Stop.

Pseudocode

```
FUNCTION findPaths(m, n, N, i, j):
  dp = 3D ARRAY[N+1][m][n] initialised to 0
  FOR step = 1 TO N:
    FOR r = 0 TO m-1:
      FOR c = 0 TO n-1:
        FOR EACH (dr, dc) IN [(-1,0),(1,0),(0,-1),(0,1)]:
          nr = r+dr; nc = c+dc
          IF nr OUT OF BOUNDS OR nc OUT OF BOUNDS:
            dp[step][r][c] += 1
          ELSE:
            dp[step][r][c] += dp[step-1][nr][nc]
  RETURN dp[N][i][j]
```

Sample Input & Output

Sample Input	m=2, n=2, N=2, i=0, j=0
--------------	-------------------------

Sample Output	6
---------------	---

Time & Space Complexity

Time Complexity	$O(N * m * n)$ — for every step, every cell is processed in constant time (4 directions).
-----------------	---

Space Complexity	$O(N * m * n)$ for the full DP table (can be reduced to $O(m * n)$ using a rolling array over steps).
------------------	---

Result

The DP program correctly computed 6 and 12 ways for the two given test cases.

Experiment 12: House Robber in a Circular Street (Circular Dynamic Programming)

Aim

To find the maximum amount of money that can be robbed from houses arranged in a circle, without robbing two adjacent houses.

Algorithm

- Start.
- Read the array nums representing money in each house arranged in a circle.
- If there is only one house, return its value.
- Since the houses are circular, the first and last house cannot both be robbed. So solve two linear (non-circular) house-robber subproblems:
 - Case A: consider houses from index 0 to n-2 (exclude last house).
 - Case B: consider houses from index 1 to n-1 (exclude first house).
- For each case, use linear DP: $\text{rob}[i] = \max(\text{rob}[i-1], \text{rob}[i-2] + \text{nums}[i])$.
- The answer is the maximum of the results of Case A and Case B.
- Stop.

Pseudocode

```
FUNCTION robLinear(nums):
```

```
    prev = 0; curr = 0
```

```
    FOR x IN nums:
```

```
        newCurr = MAX(curr, prev + x)
```

```
        prev = curr; curr = newCurr
```

```
    RETURN curr
```

```
FUNCTION robCircular(nums):
```

```
    IF LENGTH(nums) == 1: RETURN nums[0]
```

```
    RETURN MAX(robLinear(nums[0 : n-1]), robLinear(nums[1 : n]))
```

Sample Input & Output

Sample Input	nums = [2, 3, 2]
--------------	------------------

Sample Output	3
---------------	---

Time & Space Complexity

Time Complexity	$O(n)$ — two linear passes, each $O(n)$.
-----------------	---

Space Complexity	$O(1)$ extra space, since only the previous two DP values are kept at any time.
------------------	---

Result

The program correctly returned 3 for [2,3,2] and 4 for [1,2,3,1], matching the expected outputs.

Experiment 13: Climbing Stairs (Distinct Ways)

Aim

To determine the number of distinct ways to climb a staircase of n steps, taking either 1 or 2 steps at a time.

Algorithm

- Start.
- Read n , the total number of steps.
- Base cases: $\text{ways}(0) = 1$, $\text{ways}(1) = 1$.
- For each step i from 2 to n , $\text{ways}(i) = \text{ways}(i-1) + \text{ways}(i-2)$ (the ball can arrive at step i from step $i-1$ or step $i-2$).
- Return $\text{ways}(n)$.
- Stop.

Pseudocode

```
FUNCTION climbStairs(n):  
  IF  $n \leq 1$ : RETURN 1  
  prev2 = 1; prev1 = 1  
  FOR  $i = 2$  TO  $n$ :  
    curr = prev1 + prev2  
    prev2 = prev1; prev1 = curr  
  RETURN prev1
```

Sample Input & Output

Sample Input	$n = 4$
--------------	---------

Sample Output	5
---------------	---

Time & Space Complexity

Time Complexity	$O(n)$ — a single loop computing each Fibonacci-like value once.
-----------------	--

Space Complexity	$O(1)$ extra space, using only two running variables.
------------------	---

Result

The program correctly returned 5 for $n=4$ and 3 for $n=3$, matching the Fibonacci-style recurrence.

Experiment 14: Unique Paths in an m x n Grid

Aim

To count the number of unique paths a robot can take from the top-left to the bottom-right corner of an m x n grid, moving only right or down.

Algorithm

- Start.
- Read grid dimensions m and n.
- Create a 2D DP table dp of size m x n.
- Initialize the first row and first column of dp to 1 (only one way to reach any cell in the first row/column).
- For every other cell (r, c), $dp[r][c] = dp[r-1][c] + dp[r][c-1]$.
- Return $dp[m-1][n-1]$ as the total number of unique paths.
- Stop.

Pseudocode

```
FUNCTION uniquePaths(m, n):  
    dp = 2D ARRAY[m][n]  
    FOR r = 0 TO m-1: dp[r][0] = 1  
    FOR c = 0 TO n-1: dp[0][c] = 1  
    FOR r = 1 TO m-1:  
        FOR c = 1 TO n-1:  
            dp[r][c] = dp[r-1][c] + dp[r][c-1]  
    RETURN dp[m-1][n-1]
```

Sample Input & Output

Sample Input	m = 3, n = 7
--------------	--------------

Sample Output	28
---------------	----

Time & Space Complexity

Time Complexity	$O(m * n)$ — every cell of the DP table is filled exactly once.
-----------------	---

Space Complexity	$O(m * n)$ for the full table (reducible to $O(n)$ using a single rolling row).
------------------	---

Result

The program correctly returned 28 for m=3, n=7 and 3 for m=3, n=2.

Experiment 15: Intervals of Large Groups in a String

Aim

To find the [start, end] intervals of every group of 3 or more consecutive identical characters in a string.

Algorithm

- Start.
- Read the string s of length n .
- Initialize an empty result list and $\text{start} = 0$.
- Traverse the string with index i from 0 to $n-1$; whenever $s[i]$ differs from $s[i+1]$ (or i is the last index), the current group ends at i .
- If the group length $(i - \text{start} + 1)$ is ≥ 3 , append $[\text{start}, i]$ to the result list.
- Set $\text{start} = i + 1$ and continue scanning for the next group.
- Return the result list.
- Stop.

Pseudocode

```
FUNCTION largeGroups(s):  
  n = LENGTH(s)  
  result = EMPTY LIST  
  start = 0  
  FOR i = 0 TO n-1:  
    IF i == n-1 OR s[i] != s[i+1]:  
      IF (i - start + 1) >= 3:  
        result.APPEND([start, i])  
      start = i + 1  
  RETURN result
```

Sample Input & Output

Sample Input	$s = \text{"abbxxxxzzy"}$
---------------------	---------------------------

Sample Output	$[[3,6]]$
----------------------	-----------

Time & Space Complexity

Time Complexity	$O(n)$ — a single linear scan through the string.
------------------------	---

Space Complexity	$O(1)$ extra space, excluding the space required for the output list.
-------------------------	---

Result

The program correctly identified $[[3,6]]$ for the given input and an empty list $[]$ for "abc".

Experiment 16: Game of Life

Aim

To compute the next generation state of Conway's Game of Life given the current $m \times n$ board.

Algorithm

- Start.
- Read the current board of size $m \times n$ containing 0 (dead) or 1 (live) cells.
- Create a copy of the board (or encode next-state in extra bits) to avoid overwriting values needed for neighbour calculations.
- For every cell (r, c) , count the number of live neighbours among its up-to-8 surrounding cells using the original board values.
- Apply the rules: a live cell with fewer than 2 or more than 3 live neighbours dies; a live cell with 2 or 3 live neighbours survives; a dead cell with exactly 3 live neighbours becomes live.
- Store the computed next state for every cell in the new board (applied simultaneously, not in place on the original values).
- Return the new board as the next state.
- Stop.

Pseudocode

```
FUNCTION gameOfLife(board):
  m = ROWS(board); n = COLS(board)
  next = 2D ARRAY[m][n] copy of board
  FOR r = 0 TO m-1:
    FOR c = 0 TO n-1:
      liveNeighbors = COUNT live cells among 8 neighbours of (r,c) in board
      IF board[r][c] == 1 AND (liveNeighbors < 2 OR liveNeighbors > 3):
        next[r][c] = 0
      ELSE IF board[r][c] == 0 AND liveNeighbors == 3:
        next[r][c] = 1
  RETURN next
```

Sample Input & Output

Sample Input	board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
---------------------	---

Sample Output	[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
----------------------	-----------------------------------

Time & Space Complexity

Time Complexity	$O(m * n)$ — each cell's 8 neighbours are examined once, a constant-time operation per cell.
------------------------	--

Space Complexity	$O(m * n)$ for the new board that stores the next state ($O(1)$ extra if next state is encoded using extra bits in place).
-------------------------	---

Result

The next-generation board was correctly computed for both test cases, matching Conway's rules of birth, survival, and death.

Experiment 17: Champagne Tower

Aim

To determine how full the j -th glass in the i -th row of a champagne pyramid is, after pouring a given number of cups into the top glass.

Algorithm

- Start.
- Read the amount poured, and the target query_row and query_glass.
- Create a 2D DP table amount[row][glass] initialised to 0, where amount[0][0] = poured (cups poured into the top glass).
- For each row from 0 to query_row: for each glass in that row, if amount[row][glass] > 1, the excess (amount - 1) overflows equally to amount[row+1][glass] and amount[row+1][glass+1], each receiving (amount[row][glass]-1)/2.
- After processing up to query_row, the fill level of any glass is min(1, amount[row][glass]) since a glass cannot hold more than 1 cup.
- Return min(1, amount[query_row][query_glass]) as the answer.
- Stop.

Pseudocode

```
FUNCTION champagneTower(poured, query_row, query_glass):
    amount = 2D ARRAY[query_row+2][query_row+2] initialised to 0
    amount[0][0] = poured
    FOR row = 0 TO query_row:
        FOR glass = 0 TO row:
            excess = (amount[row][glass] - 1) / 2
            IF excess > 0:
                amount[row+1][glass] += excess
                amount[row+1][glass+1] += excess
    RETURN MIN(1, amount[query_row][query_glass])
```

Sample Input & Output

Sample Input	poured = 2, query_row = 1, query_glass = 1
--------------	--

Sample Output	0.50000
---------------	---------

Time & Space Complexity

Time Complexity	$O(\text{query_row}^2)$ — the DP table up to query_row has $O(\text{query_row}^2)$ cells, each processed once.
-----------------	--

Space Complexity	$O(\text{query_row}^2)$ for the 2D DP table (reducible to $O(\text{query_row})$ using a 1D rolling array).
------------------	--

Result

The program correctly computed 0.00000 and 0.50000 for the two given test cases.